LAB ASSIGNMENT – 5

Problem Statement 1: Parallel Fibonacci via OpenMP Tasks

Objective: Develop a C program that calculates the nth Fibonacci number using a recursive approach. To leverage multi-core processors, each recursive call must be encapsulated within an OpenMP **task**.

Requirements:

1. **Parallel Region:** Initialize a parallel region and ensure only a single "seed" thread starts the initial recursive call using the single directive.
2. **Tasking:** Use #pragma omp task for the recursive calls to Fib(n-1) and Fib(n-2).
3. **Synchronization:** Implement #pragma omp taskwait to ensure that the parent task waits for both child tasks to complete before summing their results.

Problem Statement 2: BST Node Summation

Obejective: Given a Binary Search Tree (BST), use nested tasks to calculate the sum of all node values. Ensure that variables are scoped correctly (shared vs private).

Aim: Practice hierarchical tasking in a non-linear data structure.

To understand the functional constraints of OpenMP synchronization by implementing a recursive sum of 10 integers and observing the behavior of #pragma omp barrier versus #pragma omp taskwait.

Problem Statement 3:

You are provided with an array of 10 integers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Your goal is to calculate the sum (55) using a **recursive divide-and-conquer** approach with OpenMP Tasks.

The core of the problem lies in the "Merge" step: a parent task splits the array into two halves, spawns child tasks to sum them, and then must **synchronize** before adding the two results together.

Experimental Versions

Version A: The Correct Tasking Model (taskwait)

In this version, use #pragma omp taskwait to synchronize the recursive calls.

- **Mechanism:** The parent task suspends execution until its immediate child tasks (the left and right sums) are completed.
- **Expected Result:** The program should output Sum: 55 reliably.
- **Student Task:** Print the Thread ID inside each task to see how different threads pick up different parts of the 10-integer array.

Version B: The Barrier Misconception (barrier)

Replace taskwait with #pragma omp barrier inside the recursive function.

- **Mechanism:** A barrier requires **all threads in the parallel team** to reach that point.
- **Expected Result: * Compile-time:** Look for errors regarding "nested barriers."
    - **Runtime:** The program will likely **deadlock (hang)**. Since some threads are stuck at a barrier waiting for others, but those "others" are busy waiting for tasks to finish, no progress is made.
- **Student Task:** If the program hangs, use Ctrl+C to terminate and explain why the threads could never "meet" at that barrier.

Version C: The Interchanged Failure (Using taskwait for Loops)

Move the logic out of recursion. Create a simple parallel for loop to populate the array with values 1–10. Use the nowait clause to remove the implicit barrier, then use taskwait instead of a proper barrier before trying to sum the array.

- **Mechanism:** taskwait only waits for *child tasks*. Since a standard loop doesn't create explicit tasks, taskwait does nothing.
- **Expected Result:** A **Race Condition**. The code may attempt to sum the array before the threads have finished writing the numbers 1–10 to it.
- **Student Task:** Check if the sum is occasionally less than 55.